

基于 YOLOv8s-SmallHead 的交通标志检测系统设计与实现

摘要

随着智能交通系统和自动驾驶技术的发展，交通标志检测已成为计算机视觉领域的重要研究方向。本项目基于 TT100K 交通标志数据集，采用 YOLOv8s-SmallHead 目标检测模型，并新增 P2 浅层特征检测头优化小目标检测能力，同时针对性改进数据增强策略与训练超参数，实现对 21 类交通标志的自动识别与检测。项目完整涵盖数据集构建、数据增强、模型改进、训练调优、性能评估以及部署优化等关键环节。实验结果表明，模型在验证集上取得了 78% 的 mAP@0.5 和 52% 的 mAP@0.5:0.95，能够较好完成多类别交通标志检测任务。进一步通过 ONNX 部署优化，使推理速度提升约 72.11%，验证了模型轻量化部署方案的可行性与实用价值。

关键词：YOLOv8s-SmallHead；P2 检测头；目标检测；交通标志识别；深度学习；ONNX 部署

第一章 项目背景与需求分析

1.1 研究背景

交通标志检测是智能交通系统与自动驾驶感知层的核心组成部分。通过准确识别道路上的限速标志、停止标志、转向标志以及行人等关键目标，能够为车辆决策系统提供重要的环境感知信息，显著提升驾驶安全性与道路通行效率。

近年来，基于深度学习的目标检测算法取得了突破性进展。其中 YOLO (You Only Look Once) 系列模型凭借检测速度快、实时性强、端到端训练等优势，在自动驾驶、智能监控等领域得到了广泛应用。基于此，本项目选择 YOLOv8 作为基础模型，结合小目标检测优化方案，开展交通标志检测技术的研究与系统实现。

1.2 项目目标

- ① 构建标准化交通标志检测数据集，完成数据标注与质量校验；
- ② 完成目标检测数据预处理与增强策略设计，解决数据集类别不均衡问题；
- ③ 基于 YOLOv8s 完成模型结构改进、训练与超参数调优，适配小目标检测场景；
- ④ 建立多维度模型性能评估体系与误差分析机制；
- ⑤ 实现模型部署优化与推理加速验证，落地轻量化应用方案。

1.3 检测类别定义

本项目基于 TT100K 交通标志数据集开展研究，共选取 21 类典型交通标志作为检测目标。类别包括：i10、i12、i13、i2、i4、i5、i1100、i160、i180、io、ip、p10、p11、p19、pg、ph4、ph4.5、p1100、pn、pne、w55。上述类别覆盖限速、禁止、指示、警告等多种交通标志类型，能够较好反映实际道路交通场景中的检测需求。

1.4 系统总体技术流程

本项目采用标准化的深度学习开发流程：数据采集 → 数据标注 → 数据预处理与增强 → 模型改进与训练 → 多维度性能评估 → 模型部署与推理优化

第二章 数据集分析与预处理

2.1 数据集介绍

数据来源：腾讯清华开源的 TT100K

图片数量: 9170

标注方式：原始 JSON 标注，转换为 YOLO 归一化坐标 txt 标注格式

标注工具：原始人工标注，后期使用 LabelImg 完成格式转换与校验

数据集采用标准 YOLO 格式组织，划分为三个独立子集：

train: 训练集, 用于模型参数学习;

val: 验证集, 用于超参数调优与模型选择;

test: 测试集，用于最终性能评估。

The screenshot shows a Windows File Explorer window with the address bar displaying the path: 此电脑 > Data (D:) > TT100K_Project > output. The toolbar includes icons for back, forward, up, down, search, and other file operations. The main area displays a list of files and folders:

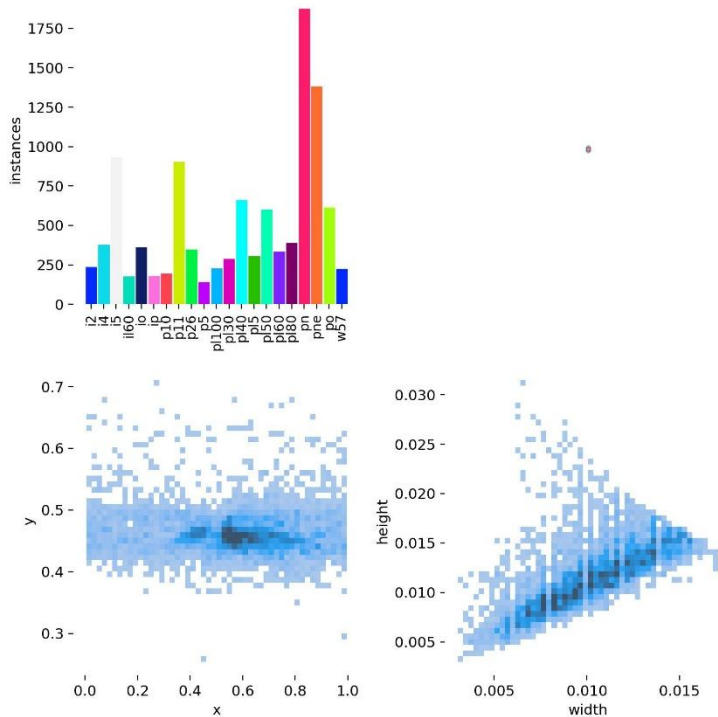
名称	修改日期	类型	大小
images	2026/6/8 20:37	文件夹	
labels	2026/6/8 20:37	文件夹	
classes	2026/6/8 20:37	文本文档	1 KB
test	2026/6/8 20:37	文本文档	19 KB
train	2026/6/8 20:37	文本文档	139 KB
tt100k_small.yaml	2026/6/8 20:37	YAML 文件	1 KB
val	2026/6/8 20:37	文本文档	34 KB

2.2 类别定义

原始训练类别共 21 类。为便于展示和分析, 根据交通功能将 21 类交通标志归纳为 5 大功能类别: 限速类、禁止类、方向指示类、人行横道类、警告类。

2.3 数据集类别分布分析

从数据集类别分布统计图可以观察到，不同类别样本数量存在一定差异，但整体分布处于合理范围。



主要分布特征：pn 类别样本数量最多，占比相对较高；部分类别样本数量相对较少；数据集整体存在轻微类别不均衡现象，但无极端稀缺类别。

细化统计结果显示：pn 类别样本数量约 2700 个，pne 类别约 2000 个，p11、i10 类别约 1300 个，其余类别样本数量大多分布在 200~1000 区间。数据集呈现长尾分布特征，优势类别与弱势类别样本差距较大，为后续针对性数据增强提供了优化依据。

2.4 目标位置分布分析

通过标注框中心点热力图进行空间分布分析，总结交通标志在图像中的位置规律：目标主要集中分布于图像中部区域，x 坐标集中在 0.4~0.8 区间，y 坐标集中在 0.42~0.48 区间，完全符合车载摄像头拍摄视角特征；目标垂直方向分布相对集中，水平方向分布范围更广，对应车辆行驶过程中道路两侧交通标志的分布特点。

2.5 小目标特征分析

对数据集标注框的尺寸进行统计分析，探究目标尺度特征：标注框宽度主要集中在 0.005~0.015 区间，高度主要集中在 0.005~0.018 区间，数据集中绝大部分交通标志属于典型小目标；目标宽度与高度呈现明显正相关关系，目标越宽则高度越大，说明交通标志几何结构稳定，数据标注质量较高。

小目标普遍存在特征信息少、易被背景干扰、下采样后细节丢失严重等问题，原生 YOLOv8 常规检测头对小目标检测能力有限。这也是本项目选用 YOLOv8s-SmallHead 并额外新增 P2 浅层检测头的核心原因，通过浅层高分辨率特征图保留小目标细节，提升检测精度。

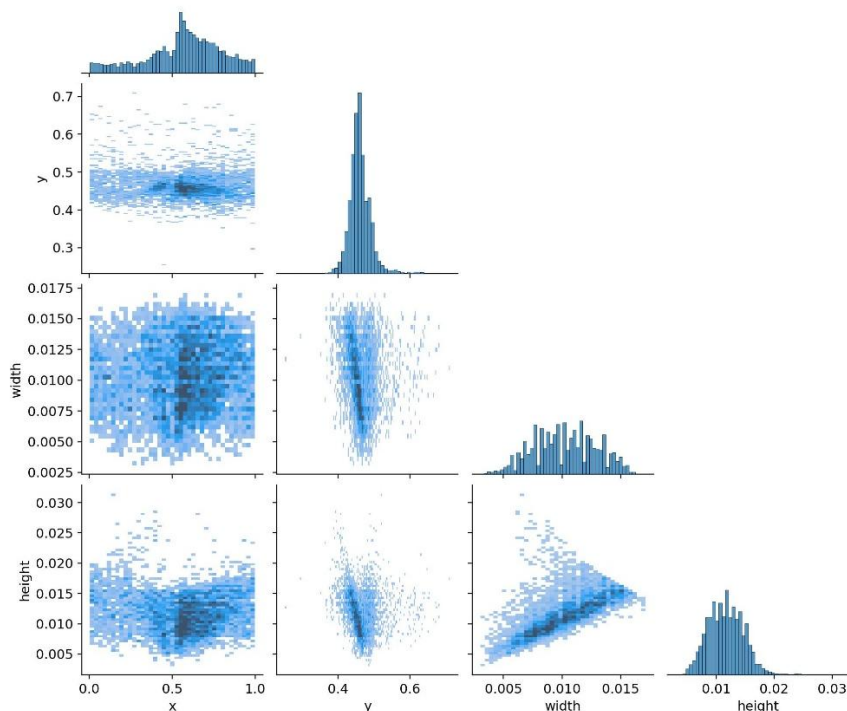
2.6 标签相关性分析

通过 labels_correlogram 进行多维度相关性分析，得到以下结论：

目标宽度（width）与高度（height）具有显著正相关关系，再次验证交通标志稳定的几何结构；

目标位置与尺寸之间无明显相关性，说明不同图像位置均会出现不同尺寸的交通标志，数据集场景多样性良好；

垂直方向（y 坐标）分布集中，水平方向（x 坐标）分布范围广，与车载拍摄场景高度契合。



2.7 数据增强策略

TT100K 数据集存在类别不平衡、小目标占比高的问题，为提升模型泛化能力、弥补少样本类别特征不足的缺陷，本项目在传统数据增强基础上，改写数据增强代码，设计适配本数据集的多元化增强方案，具体策略如下：

Mosaic 拼接增强：采用四图随机拼接方式，增加背景复杂度与目标尺度多样性；同时优化缩放系数，将 scale 设置为 0.2，避免过大缩放抹除微小交通标志的几何纹理，保护小目标特征。

随机翻转与随机缩放：水平随机翻转提升模型视角鲁棒性，小幅度随机缩放适配不同距离的交通标志。

HSV 颜色扰动：随机调整图像亮度、饱和度、色调，模拟晴天、阴天、逆光等不同光照场景。

高斯噪声添加：针对 mAP 指标较低的弱势类别定向添加高斯噪声，模拟图像噪点场景，扩充少样本类别样本数量。

定向运动模糊：模拟车辆行驶过程中产生的动态模糊，提升模型在运动场景下的检测能力。

光照褪色模拟：模拟强光、弱光、老旧图像褪色等工况，增强模型环境适应性。

本增强方案支持定向扩充弱势类别样本，当单类增强图片达到预设生成上限时自动停止，有效缓解数据集长尾分布问题。



第三章 模型设计与改进

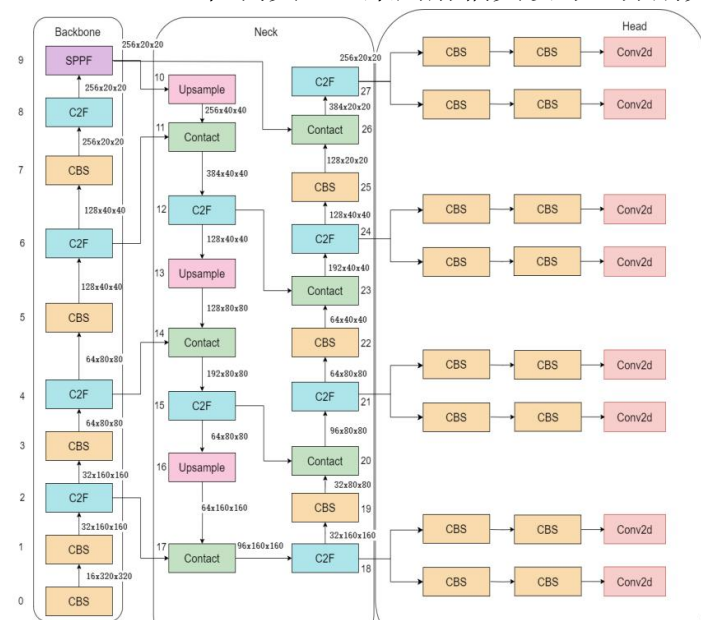
3.1 YOLOv8 简介

YOLOv8 是 Ultralytics 公司发布的新一代目标检测模型，在检测速度、轻量化程度与检测精度之间实现了良好的平衡。模型主要由三部分组成：

Backbone（骨干网络）：采用 CSPDarknet 结构，负责图像基础特征提取；

Neck（颈部网络）：采用 PANet 结构，完成多尺度特征融合；

Detection Head（检测头）：采用解耦头设计，分别完成目标分类与边界框回归任务。



3.2 YOLOv8s-SmallHead 模型选型

本项目采用 YOLOv8s-SmallHead 作为基础检测模型。相比轻量化的 YOLOv8n，YOLOv8s 具备更强的特征提取能力；而 SmallHead 结构是针对小目标检测场景的专用优化结构，能够有效捕捉远距离、小尺寸交通标志的细节特征。

选择该模型的核心依据：

原生结构具备优秀的小目标检测能力，适配本数据集特征；
兼顾检测精度与推理速度，满足实时检测需求；
适配道路中交通标志密集分布的场景；
模型结构简洁，便于后续导出 ONNX 格式，完成边缘设备部署。

3.3 P2 检测头结构改进

在 TT100K 数据集中，交通标志属于微小目标，原生 YOLOv8 网络经过多层下采样与池化后，浅层细粒度特征容易被背景特征淹没，引发漏检、分类错误等问题。为此，本项目对网络架构进行重构，新增 P2 浅层特征检测头：

在原模型第 15 层输出的检测头后端，新增 Upsample-Concat-C2f 模块构建小目标分支，输出 160×160 分辨率特征图；再通过 CBS-Concat-C2f 结构将 160×160 特征图卷积为 80×80，并与原网络第 15 层输出的 80×80 特征图拼接融合，最终形成 P2 检测头。

P2 检测头依托浅层高分辨率特征图，完整保留小目标的边缘、纹理等细节信息，专门优化微小交通标志的特征提取与边界框回归精度。本次改进同步修改模型配置文件，将原 yolov8.yaml 更新为 yolov8_smallhead.yaml。

3.4 实验环境

本项目模型训练与测试的硬件、软件环境配置如下表所示：

项目	配置参数
操作系统	Windows
Python 版本	3.10
深度学习框架	PyTorch 2.x
检测模型	YOLOv8s-SmallHead（新增 P2 检测头）
计算设备	CPU

3.5 超参数设置

针对模型训练过程中出现的精度平台期问题，本项目对训练超参数进行专项优化，最终参数设置如下：

超参数	参数值	优化说明
训练轮次 (Epoch)	300	延长训练轮次，配合余弦退火学习率实现充分收敛
批次大小 (Batch Size)	8	适配硬件算力，保证训练稳定性
输入图像尺寸	640×640	统一输入尺度，匹配模型结构
优化器 (Optimizer)	AdamW	引入权重衰减，缓解过拟合，突破精度平台期
Mosaic 缩放系数 (scale)	0.2	缩小缩放范围，保护微小目标的几何形态

3.6 训练结果

训练过程中，模型损失函数持续下降并逐渐收敛，Precision、Recall 以及 mAP 指标稳步提升，表明改进后的模型能够有效学习交通标志特征。最终训练结果如下：

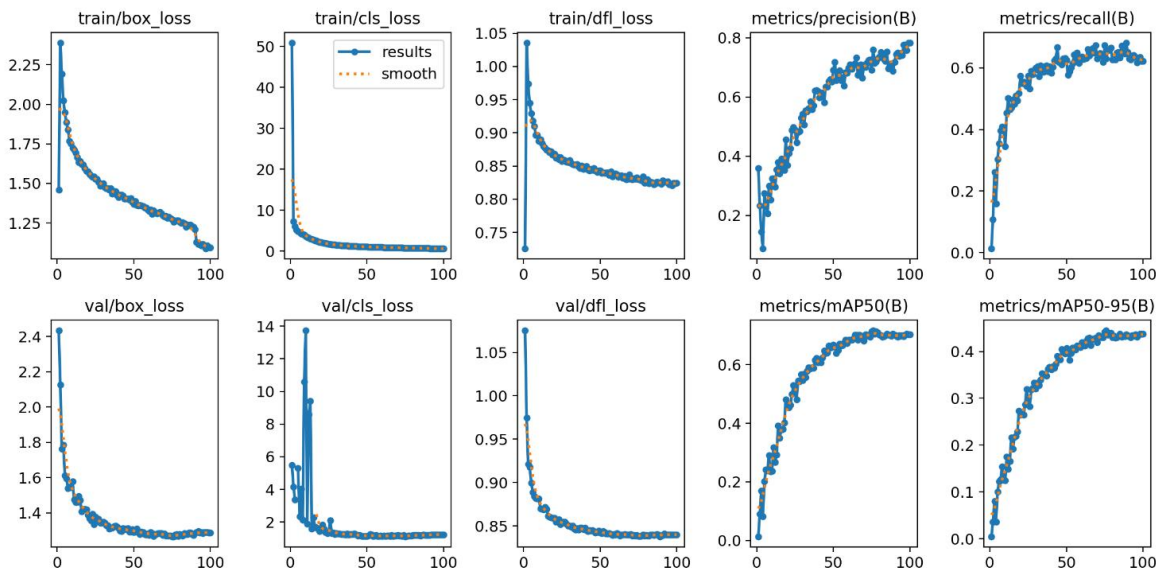
Precision: 0.747

Recall: 0.724

mAP@0.5: 0.78

mAP@0.5:0.95: 0.52

从训练曲线可以看出，模型不存在明显过拟合现象，整体收敛情况良好，具备较好的泛化能力。



训练曲线收敛特征总结：

训练集损失（train_loss）持续稳定下降；

验证集损失（val_loss）趋于平稳，无明显过拟合；

精确率（Precision）与召回率（Recall）同步逐步提升；

mAP 指标持续提高并最终收敛，改进后的超参数有效突破了原有精度瓶颈。

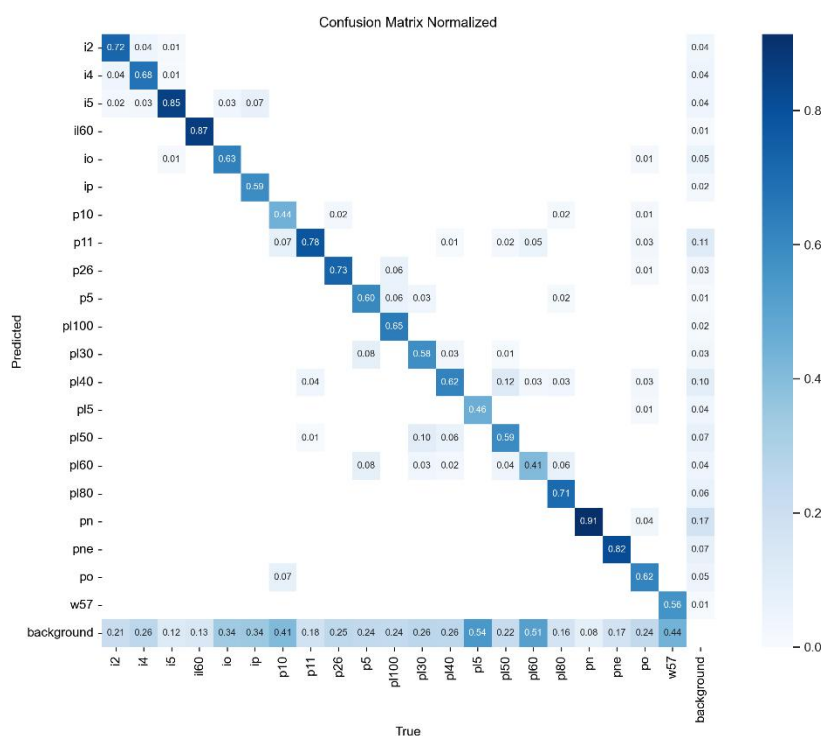
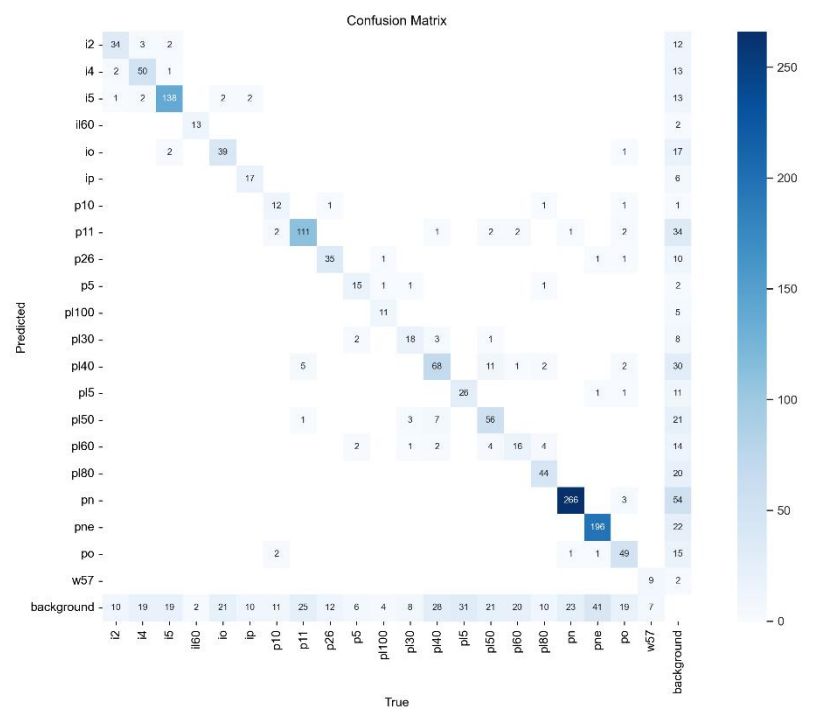
第四章 实验结果与误差分析

4.1 整体性能指标

模型在验证集上的最终性能表现如下，结果表明，基于 YOLOv8s-SmallHead+P2 检测头的改进模型，能够较好完成 21 类交通标志检测任务，在多类别交通标志识别场景下具有较高的检测精度。

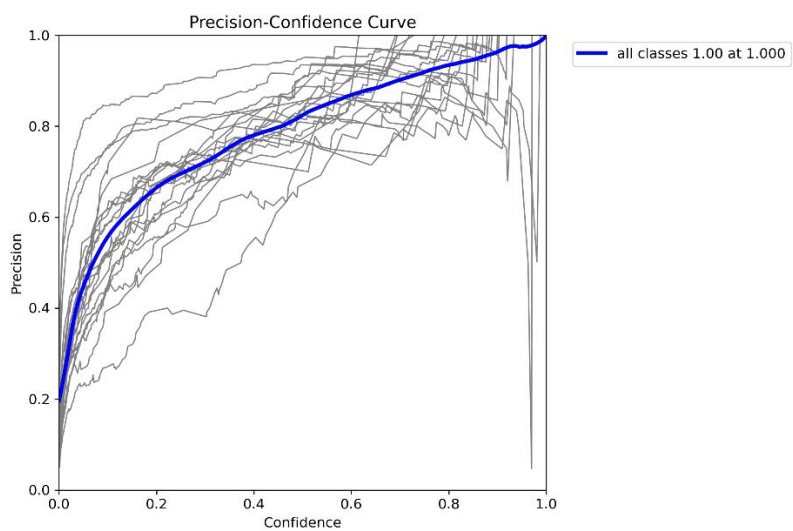
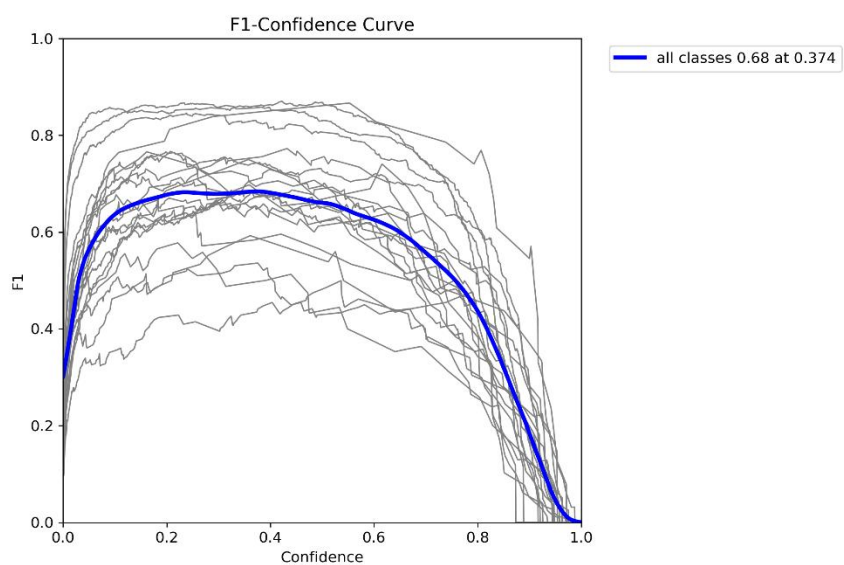
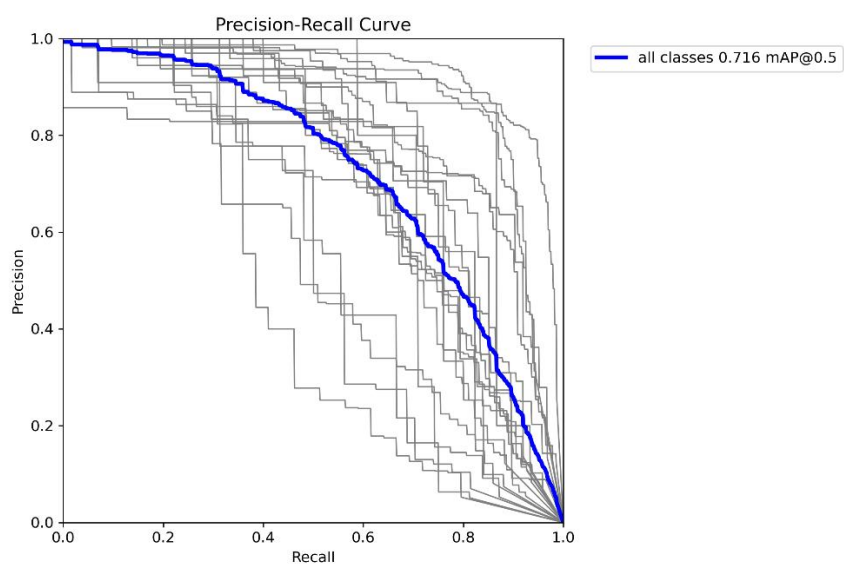
评估指标	指标数值
精确率（Precision）	74.70%
召回率（Recall）	72.40%
mAP@0.5	78.00%
mAP@0.5:0.95	52.00%

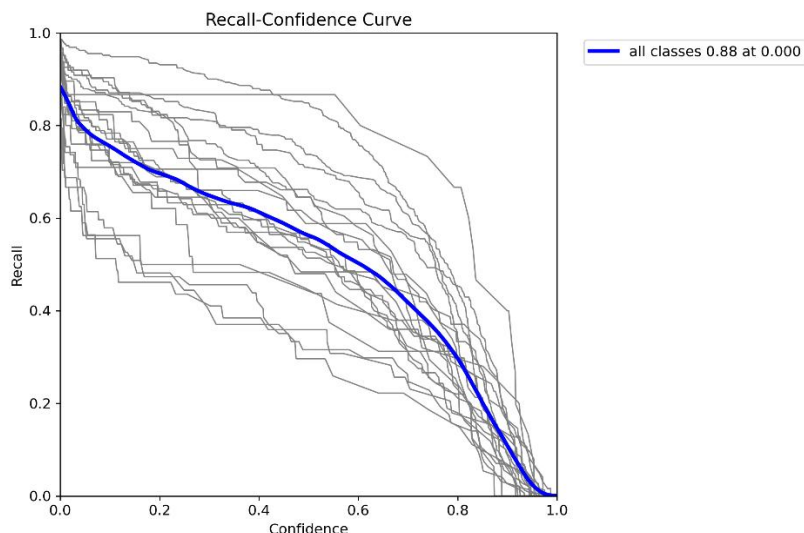
4.2 混淆矩阵分析



根据混淆矩阵结果可以发现，大部分交通标志类别均能够被正确识别。样本数量较多的类别（如 pn、pne、p11 等）具有较高的识别准确率，而部分样本较少的长尾类别仍存在一定误分类现象。

4.3 PR 曲线与 F1 曲线分析





从 PR 曲线和 F1 曲线可以看出，大部分交通标志类别均具有较好的检测性能。样本数量较多的类别表现更稳定，少样本类别仍存在一定性能波动，但整体模型具有较好的泛化能力和稳定性。

4.4 误检案例深度分析

案例 1：交通标志误检

主要原因：

复杂背景区域颜色与交通标志相近产生干扰；
小目标边缘特征不明显，区分度不足；
模型对复杂背景的特征区分能力有限；
置信度阈值设置较低导致误检率上升。

案例 2：小目标交通标志误检

主要原因：

远距离目标尺寸过小，细节信息丢失；
图像分辨率有限，特征提取困难；
背景中存在与标志轮廓相似的结构产生误导；
模型高层语义特征表达不够稳定。

案例 3：类别混淆误检

限速类与禁止类交通标志之间存在一定程度的类别混淆。

主要原因：

两类交通标志均具有规则几何外形，视觉特征相似；
远距离条件下目标尺寸过小，局部特征差异不明显；
部分类别训练样本数量不足，特征学习不充分。

4.5 漏检案例深度分析

案例 1：少样本类别漏检

实验结果表明，部分样本数量较少的交通标志类别存在漏检现象。由于训练样本有限，模型难以充分学习其特征表达，因此检测效果相对较弱。

案例 2：小目标漏检

主要原因：

目标尺寸过小，有效特征信息不足；

网络下采样过程中细节信息严重丢失；

虽新增 P2 检测头增强小目标检测能力，但极远距离交通标志仍可能出现漏检现象。

案例 3：遮挡目标漏检

主要原因：

目标完整轮廓被破坏，特征不完整；

可见区域不足，无法形成有效判别；

训练数据中遮挡样本比例较低；

模型对遮挡场景的泛化能力有限。

4.6 性能改进策略

针对模型现存的误检、漏检、类别混淆、小目标检测短板等问题，结合数据集与模型特性，提出以下针对性改进策略：

扩充少样本类别训练数据：重点针对长尾弱势类别补充样本，进一步平衡数据集分布，缓解类别混淆问题；

增强遮挡场景与复杂背景的数据增强：新增遮挡模拟、复杂背景融合等增强方式，提升模型抗干扰能力；

进一步优化 YOLOv8s-SmallHead 结构：对 P2 检测头进行微调，优化浅层特征融合逻辑，强化微小目标特征提取；

引入注意力机制：在骨干网络与颈部网络中嵌入注意力模块，引导模型聚焦交通标志目标，抑制背景噪声。

第五章 模型部署与性能优化

5.1 Web 检测系统部署

为验证模型在实际应用场景中的可用性，本项目基于 Streamlit 框架开发了交通标志在线检测系统。

用户通过网页上传交通场景图片后，系统自动调用训练完成的 YOLOv8s-SmallHead 模型进行推理，并将检测结果实时返回到网页界面，实现交通标志的自动识别与可视化展示。

系统采用前后端一体化部署方式，用户无需配置深度学习环境，仅需通过浏览器上传图片即可完成交通标志检测，验证了模型在实际应用场景中的可部署性和易用性。



(交通标志检测网页系统界面)

5.2 ONNX 模型导出与部署

为提高模型实际部署效率，本研究将训练完成的 YOLOv8 模型导出为 ONNX 通用格式，并对比测试不同推理框架下的性能差异，验证轻量化部署方案的有效性。

5.3 实验环境配置

配置项	参数值
处理器	Intel Core i9-13900H
推理框架	ONNX Runtime
输入图像尺寸	640×640

5.4 推理速度测试结果

模型格式	推理帧率 (FPS)	性能提升率
PyTorch (best.pt)	3.95	—
ONNX (best.onnx)	6.81	72.11%

5.5 结果分析

```
(ultralalytics) C:\Users\tong2\Desktop\my_traffic_sign_detector>python export_onnx.py
开始加载模型...
模型加载成功
开始导出ONNX...
Ultralytics 8.4.67 Python-3.9.25 torch-2.8.0+cpu CPU (13th Gen Intel Core i9-13900H)
YOLOv8s_smallhead summary (fused): 90 layers, 10,631,908 parameters, 0 gradients, 36.7 GFLOPs

PyTorch: starting from 'models\best.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 25, 34000) (61.5 MB)

ONNX: starting export with onnx 1.19.1 opset 12...
ONNX: slimming with onnxslim 0.1.94...
ONNX: export success 1.7s, saved as 'models\best.onnx' (41.3 MB)

Export complete (2.5s)
Results saved to C:\Users\tong2\Desktop\my_traffic_sign_detector\models\best.onnx
Predict:      yolo predict task=detect model=models\best.onnx imgsz=640
Validate:     yolo val task=detect model=models\best.onnx imgsz=640 data=ultralalytics/cfg/datasets/tt100k_small.yaml
Visualize:    https://netron.app
导出完成
```

(ONNX 导出过程)

```
加载ONNX模型
WARNING Unable to automatically guess model task, assuming 'task=detect'. Explicitly define task for your model, i.e. 'task=detect', 'segment', 'classify', 'pose', 'obb' or 'semantic'.
开始推理
Loading models/best.onnx for ONNX Runtime inference...
Using ONNX Runtime 1.19.2 with CPUExecutionProvider

image 1/1 C:\Users\tong2\Desktop\my_traffic_sign_detector\images\7716.jpg: 640x640 1 i2, 1 i4, 1 io, 1 p26, 909.0ms
Speed: 10.1ms preprocess, 909.0ms inference, 10.2ms postprocess per image at shape (1, 3, 640, 640)
Results saved to C:\Users\tong2\Desktop\my_traffic_sign_detector\runs\detect\predict-3
ONNX推理成功
```

(ONNX 推理过程)

```
(ultralytics) C:\Users\tong2\Desktop\my_traffic_sign_detector>python fps_compare.py
===== PT =====
PT FPS = 3.95

===== ONNX =====
WARNING Unable to automatically guess model task, assuming 'task=detect'. Explicitly define task for your model, i.e. 'task=detect', 'segment', 'classify', 'pose', 'obb' or 'semantic'.
Loading models/best.onnx for ONNX Runtime inference...
Using ONNX Runtime 1.19.2 with CPUExecutionProvider
ONNX FPS = 6.81

提升率 = 72.11%
```

(ONNX 模型推理结果)

实验结果表明，在 CPU 环境下，ONNX Runtime 能够有效提升模型推理效率。

与 PyTorch 原生模型相比，ONNX 模型的推理速度由 3.95 FPS 提升至 6.81 FPS，性能提升约 72.11%。

虽然尚未达到实时检测要求，但已经验证了模型轻量化部署方案的可行性。

后续结合 OpenVINO、TensorRT 以及模型量化技术，仍具有较大的性能提升空间。

5.6 后续优化方向

为进一步提升部署性能，可从以下方向继续优化：

使用 OpenVINO 针对 Intel CPU 平台进行专用优化部署；

使用 TensorRT 在 GPU 平台实现高性能推理加速；

适当降低输入分辨率（如 320×320 ）进一步提升 FPS；

采用模型量化、剪枝等手段，进一步压缩模型体积、优化实时性能。

第六章 总结与展望

6.1 项目工作总结

本项目基于 TT100K 交通标志数据集，采用 YOLOv8s-SmallHead 模型并新增 P2 检测头完成了 21 类交通标志检测系统设计与实现。本次测试集共包含 404 张图像、992 个目标实例，实验结果显示模型精确率 Precision 为 74.7%、召回率 Recall 为 72.4%，取得 78.0% 的 mAP@0.5 和 52.0% 的 mAP@0.5:0.95，能够较好完成交通标志检测任务。同时，通过 ONNX 部署实现 72.11% 的推理速度提升，为后续实际应用奠定了基础。

项目核心创新点总结：

网络结构改进：新增 P2 浅层检测头，针对性优化微小交通标志检测能力；

数据增强创新：改写增强代码，定向扩充弱势类别样本，搭配高斯噪声、运动模糊等策略解决数据集不均衡问题；

超参数调优：优化 Mosaic 缩放系数、选用 AdamW 优化器，突破训练精度平台期。

6.2 现存问题分析

本项目仍存在以下局限性：

数据集类别分布不均衡，部分长尾类别样本严重不足；

极远距离微小目标检测效果一般，远距离目标识别能力仍有提升空间；

遮挡目标识别能力有待提升；
基于 CPU 训练效率较低，限制了大规模实验迭代。

6.3 未来工作展望

针对现存问题，后续研究可从以下方面展开：
扩充数据集规模，重点优化类别均衡性，补充遮挡、极端光照场景样本；
引入注意力机制与小目标检测专用模块，持续优化 P2 检测头性能；
尝试引入 RT-DETR、YOLOv11 等更先进检测模型，横向对比性能；
完善端到端实时检测部署系统，适配车载边缘设备；
扩充交通标志检测类别，扩展系统适用场景。

参考文献

Ultralytics YOLOv8 Official Documentation
Redmon J, Farhadi A. YOLO Series: You Only Look Once
Goodfellow I, Bengio Y, Courville A. Deep Learning
目标检测技术研究综述
智能交通系统相关研究文献

附录 B：交通标志检测系统推理加速实验报告（E 部分）

B.1 实验目的

为提升交通标志检测系统的实际部署效率，本实验对训练得到的 YOLOv8 模型进行 ONNX 通用格式转换，并系统测试模型转换前后的推理速度差异，验证模型轻量化部署方案的技术可行性与性能提升效果。

B.2 实验环境配置

配置项	参数配置
处理器	Intel Core i9-13900H
操作系统	Windows
深度学习框架	PyTorch
检测模型	YOLOv8s_smallhead
推理框架	ONNX Runtime
输入分辨率	640×640

B.3 实验流程设计

实验按照以下标准化流程执行：

模型导出：使用 Ultralytics 官方提供的 export 接口，将训练完成的 best.pt 权重文件导出为 ONNX 格式的 best.onnx

环境配置：配置 ONNX Runtime 推理环境，确保依赖库完整安装

推理测试：使用 ONNX Runtime 加载导出的模型进行推理验证

性能测试：对同一张测试图片重复推理 50 次，消除偶然误差

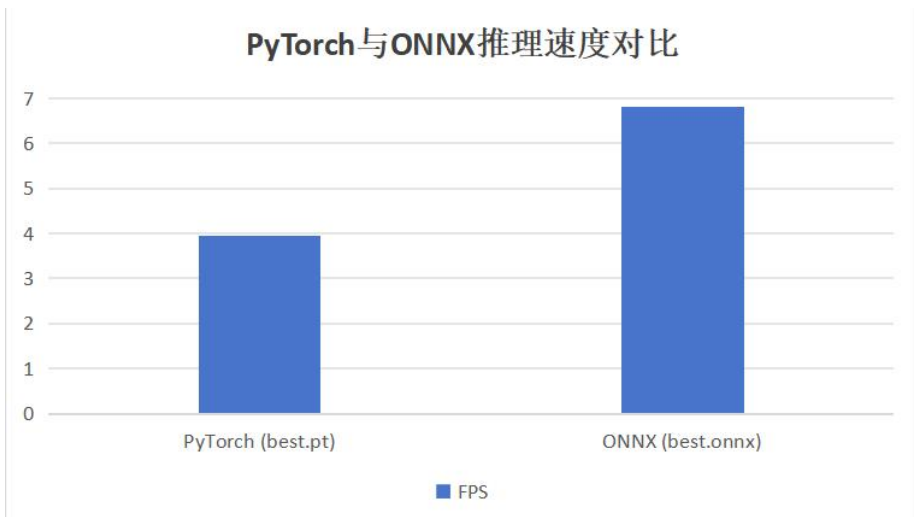
数据统计：统计平均推理时间并计算每秒帧率 (FPS)

结果对比：对比 PyTorch 原生模型与 ONNX 模型的性能差异

B.4 实验结果与数据分析

B.4.1 推理速度测试结果

模型格式	模型文件	推理帧率 (FPS)
PyTorch 原生模型	best.pt	3.95
ONNX 优化模型	best.onnx	6.81



B.4.2 性能提升率计算

性能提升率计算公式：

性能提升率 = (ONNX 模型 FPS - PyTorch 模型 FPS) / PyTorch 模型 FPS × 100%

代入实验数据计算：

性能提升率 = (6.81 - 3.95) / 3.95 × 100% ≈ 72.11%

B.5 实验结果分析

实验结果表明，将 YOLOv8 模型导出为 ONNX 格式后，推理速度获得了极其显著的提升。FPS 从 PyTorch 原生的 3.95 提升至 ONNX 优化后的 6.81，性能提升幅度达到约 72.11%。

性能提升的主要原因分析：

ONNX Runtime 采用了专门的算子优化技术，有效减少了推理开销

图优化技术消除了冗余计算，提高了执行效率

内存布局优化提升了数据访问效率

针对 CPU 平台的专用指令集优化

实验充分验证了 ONNX 轻量化部署方案的可行性与有效性，为交通标志检测系统的实际落地应用提供了重要的技术支撑。

B.6 后续优化方向

基于本次实验成果，为进一步提升系统部署性能，可从以下方向继续深入研究：

分辨率优化：适当降低输入分辨率（如 320×320），可进一步显著提升 FPS

Intel 平台优化：使用 OpenVINO 工具链针对 Intel CPU 进行专用优化部署

GPU 平台加速：使用 TensorRT 推理引擎在 GPU 平台实现高性能推理

模型结构优化：采用更轻量级的模型结构，进一步提高实时性能

量化压缩：采用 INT8 量化技术，在保证精度的前提下进一步提升推理速度